



Паттерн «Стратегия» (Strategy)

Группа: Поведенческие паттерны • Выполнил: Гриненко Никита



Проблема — «IF-ELSE»

Ситуация

Один объект должен выполнять действие разными способами (навигация: авто / пешком / транспорт).

Последствия

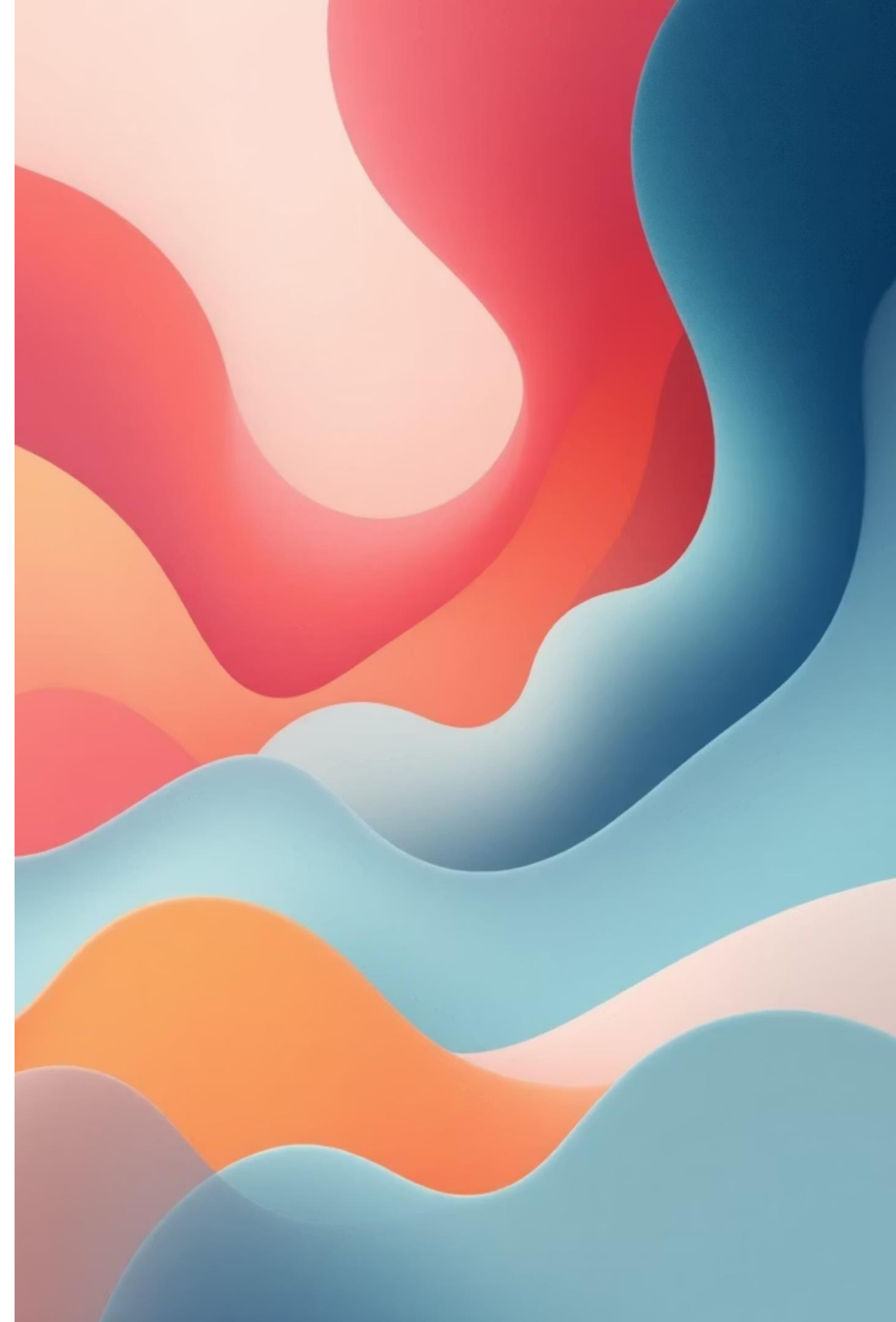
Огромные методы со switch/if-else: код трудно читать, тестировать и расширять; изменение одного алгоритма рискует сломать другие.

Решение

Паттерн «Стратегия» предлагает вынести варианты алгоритма в отдельные классы с общим интерфейсом и хранить ссылку на него в контексте — объект делегирует выполнение выбранной стратегии.

Шаги

Выделить классы-алгоритмы → Создать общий интерфейс → Контекст держит ссылку и делегирует.



Аналогия: Путешествие в аэропорт



Компоненты

- Контекст: путешественник
- Цель: добраться до аэропорта
- Стратегии: автобус (дешево/медленно), такси (быстро/дорого), велосипед (бесплатно/тяжело)
- Выбор зависит от денег, времени и погоды, выбирает сам путешественник

Структура (UML) — 3 ключевых компонента



Strategy (интерфейс)

Общий метод execute() / deliver().



ConcreteStrategies

Конкретные реализации алгоритмов (Алгоритм А, Б, В).



Context

Хранит ссылку на Strategy и имеет setStrategy() для смены поведения.



Пример

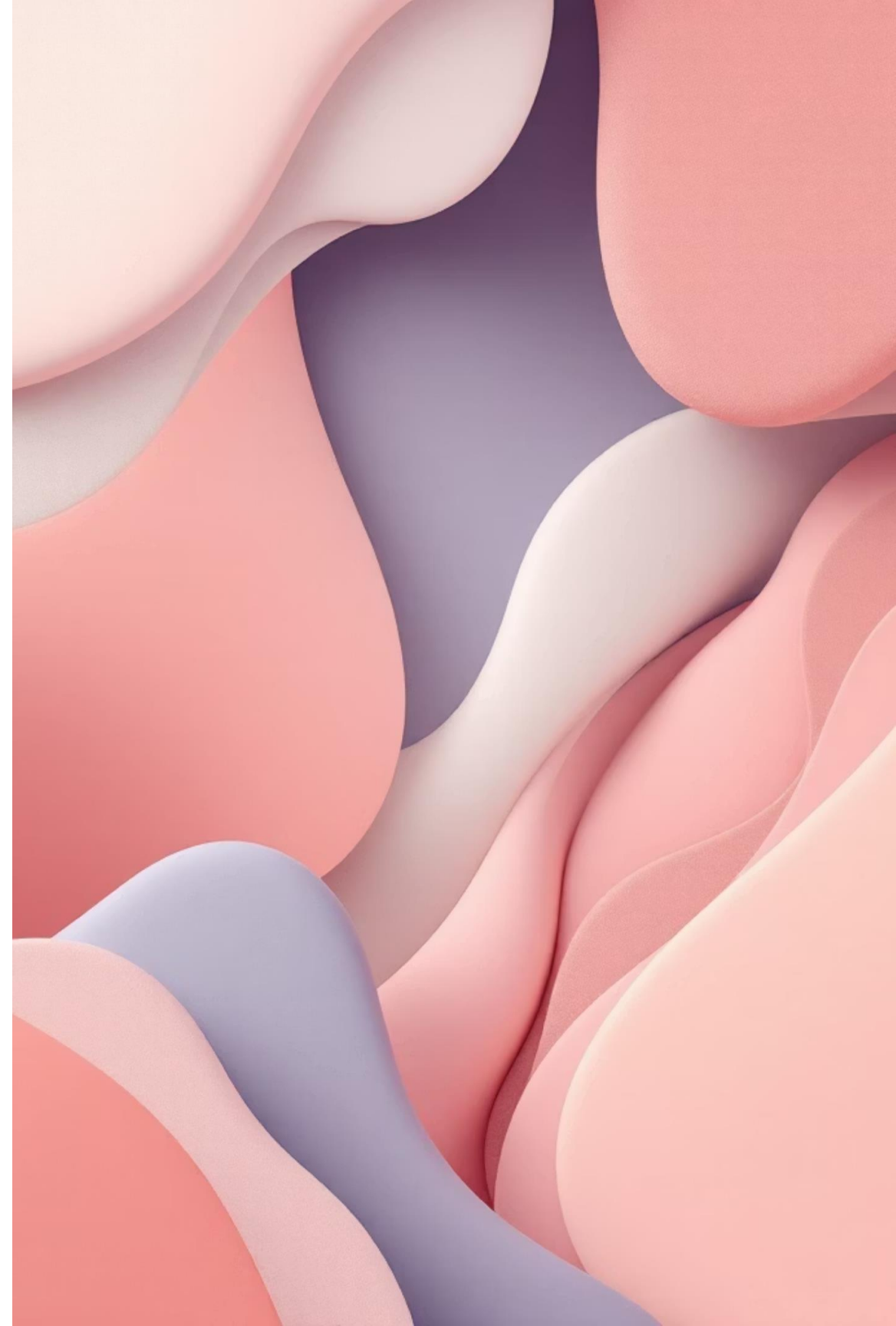
Интерфейс Strategy и две реализации; Context делегирует вызов выбранной стратегии.

```
# Интерфейс
class Strategy:
    def deliver(self, package): pass

# Конкретные реализации
class CourierStrategy(Strategy):
    def deliver(self, package): return f"Доставка {package} курьером"

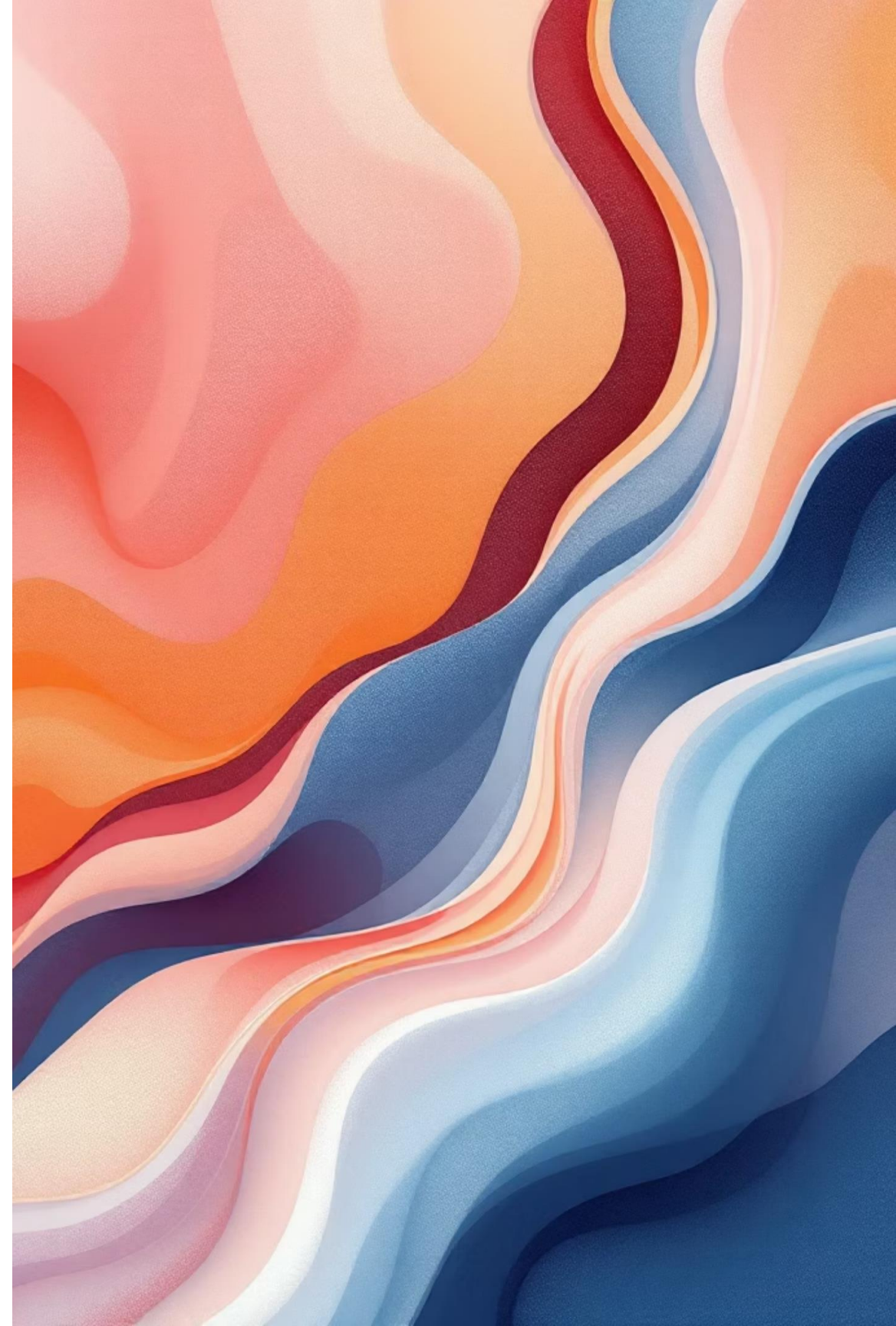
class DroneStrategy(Strategy):
    def deliver(self, package): return f"Доставка {package} дроном"

# Контекст
class DeliveryService:
    def __init__(self, strategy): self.strategy = strategy
    def send(self, item): print(self.strategy.deliver(item))
```



Когда применять?

- Множество похожих классов, отличающихся поведением.
- Нужно скрыть реализацию алгоритмов от клиента.
- Требуется менять поведение во время выполнения.
- В коде много условных операторов для выбора алгоритма.





Плюсы и минусы

Плюсы

- Легко добавлять новые алгоритмы.
- Изоляция алгоритмов от контекста.
- Замена поведения «на лету».

Минусы

- Усложнение кода из-за новых классов.
- Клиент должен понимать различия стратегий, чтобы выбрать нужную.



Заключение

Паттерн «Стратегия» делает код гибким и модульным: вместо вшивания логики в класс — превращаем её в сменный модуль, упрощающий расширение и тестирование.



Ключ: разделять вариативность алгоритма и хранить единую цель в контексте.